

LABORATORIO 12

El panel del administrador

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo.

Introducción

1 · El tema

En este laboratorio no aparecen conceptos nuevos. Se arma una herramienta. Son ocho preguntas que quien administra una plataforma de datos corre cada mañana para saber si sus tablas están sanas antes de que alguien reclame.

Las ocho miran una sola tabla, `panel_lab12`, que el paso 0 arma con problemas puestos a propósito. Cada problema es lo que va a encontrar una de las ocho preguntas.

2 · El problema, en el almacén

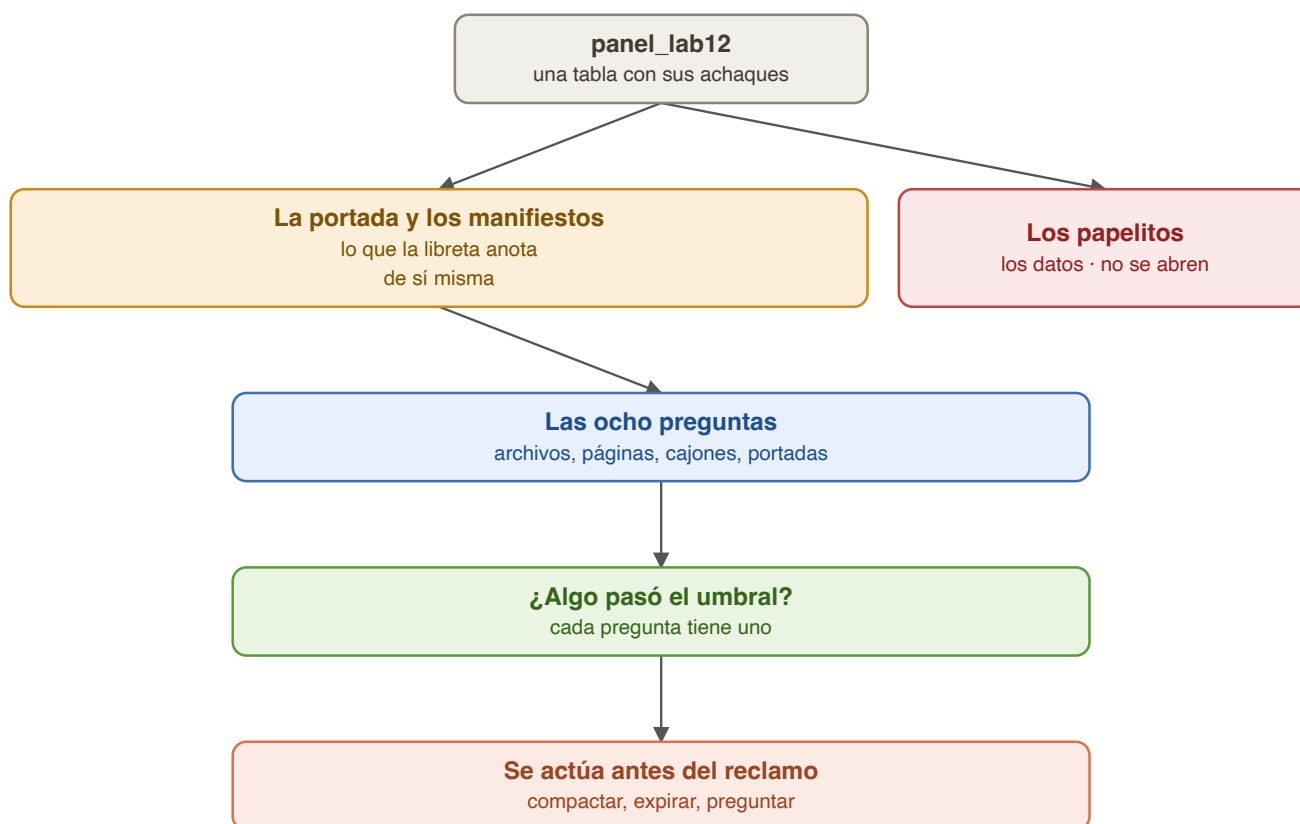
El almacén funciona todos los días. Llegan cargas grandes y chicas, se corrigen renglones, se borran anotaciones, se agrega una columna nueva a la libreta. Con el tiempo la libreta se llena de papelitos chicos, de papelitos viejos que ya nadie usa y de cajones muy desparejos. Nada de eso se ve desde el mostrador. El almacén sigue atendiendo, pero cada vez más lento, hasta que un día un cliente reclama.

3 · El problema en términos técnicos

Una tabla Iceberg que trabaja todos los días acumula archivos chicos, páginas viejas que siguen ocupando disco, particiones desbalanceadas y cambios de estructura. Ninguna de esas cosas produce un error. Las consultas siguen funcionando, solo que cada vez más lentas y la tabla cada vez más pesada. Si nadie mira, el problema aparece cuando ya es caro.

Mirar abriendo los datos no sirve, porque en una plataforma grande abrir todos los archivos de todas las tablas cada mañana cuesta más que el problema.

4 · El diagrama



panel_lab12 (gris). Es la tabla del día. El paso 0 la carga con achaques a propósito, para que cada pregunta encuentre algo.

La portada y los manifiestos (amarillo). Toda tabla Iceberg anota de sí misma cuántos papelitos tiene, cuánto pesa cada uno, qué páginas hay y cuándo se escribió cada una. Eso vive en la portada, que es el `metadata.json`, y en los manifiestos. Es poco y está a mano.

Los papelitos (rojo). Son los archivos Parquet con los datos. Ninguna de las ocho preguntas los abre.

Las ocho preguntas (azul). Son ocho consultas cortas sobre las vistas de sistema de la tabla, que leen la portada y los manifiestos.

¿Algo pasó el umbral? (verde). Cada pregunta devuelve un número, y cada número se compara con un umbral. Los umbrales de esta guía son de ejemplo.

Se actúa antes del reclamo (coral). Si un número pasó su umbral, se actúa. Se compacta, se botan páginas viejas o se va a preguntar quién hizo qué.

5 · La solución

En el almacén, el dueño no revisa la mercadería caja por caja cada mañana. Mira la libreta, que ya tiene anotado cuántos papelitos hay, cuánto pesa cada uno y cuándo se escribió cada página. Con eso sabe en un minuto dónde está el desorden.

En Iceberg eso son las **vistas de sistema**. `.files`, `.all_files`, `.snapshots`, `.partitions` y `.metadata_log_entries` muestran lo que la tabla anota de sí misma. Consultarlas no lee datos, así que se pueden correr todas las mañanas sobre todas las tablas sin que a nadie le moleste. Eso convierte un informe mensual en un panel diario.

6 · Los pasos

- **Paso 0.** Armas `panel_lab12` con sus achaques, un mes gordo, veintidós cargas chicas, tres correcciones, una columna nueva y un borrado.
- **Paso 1.** Cuántos archivos tiene la tabla y cuánto pesan, y cómo se extiende la pregunta a varias tablas.
- **Paso 2.** Cuántos de esos archivos son chicos.
- **Paso 3.** Cuántas páginas acumula la tabla y desde cuándo.
- **Paso 4.** Cuánto ocupan los archivos que ya no cuentan.
- **Paso 5.** Si hay particiones desaparejas.
- **Paso 6.** Cuándo fue la última escritura y qué hizo.
- **Paso 7.** Cuántas versiones de portada hay y si cambió la estructura.
- **Paso 8.** Si la tabla se compactó alguna vez.
- **Cierre.** Las ocho preguntas con lo que salió y un umbral de ejemplo para cada una.

Si quieres repetir el laboratorio desde cero, borra la tabla con el comando de abajo, desde la carpeta del repositorio. El paso 0 igual empieza con un `DROP TABLE IF EXISTS`.

```
bash
bin/reiniciar-lab.sh 12
```

Paso 0 · La tabla de hoy, con sus achaques



Enero de un tirón (azul). La primera carga trae todo enero de una vez. Así empiezan casi todas las tablas, con el histórico que ya existía. Queda un cajón mucho más gordo que los demás.

22 cargas diarias (morado). De febrero a diciembre se cargan dos días por mes, el 10 y el 20, cada uno en una escritura aparte. Es una ingesta diaria en miniatura, y deja muchos papelitos chicos.

3 correcciones (amarillo). Tres `UPDATE` sobre enero. Cada corrección que encuentra filas escribe un papelito nuevo y deja el viejo en el suelo.

Columna nueva (verde agua). Se agrega `revisado_por`. Cambia la estructura sin tocar un dato.

Un borrado (rojo). Se borran los documentos del 20 de julio.

panel_lab12 (gris). Con los cinco achaques puestos, la tabla queda como una que lleva un tiempo en producción.

Celda 0.1 · Entrar a tu espacio

La pregunta. ¿En qué espacio de trabajo vas a crear la tabla?

Por qué ahora. Todas las celdas que siguen nombran la tabla sin el espacio adelante, y eso funciona solo si estás parado en el tuyo.

En el almacén. Es pararse frente a tu propio estante antes de empezar a anotar.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Todo lo que va después de dos guiones lo ignora el motor.
- `USE mi_espacio` te deja parado en tu espacio de trabajo, `mi_espacio`.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:09:50 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La sentencia funcionó. `Listo. La sentencia se ejecutó.` es lo que escribe el cuaderno cuando una sentencia no devuelve filas.

Las tres primeras líneas no son errores. Salen solo en la primera celda que usa Spark, cuando arranca.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:09:50` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.

Celda 0.2 · Partir de cero

La pregunta. ¿Hay una `panel_lab12` de una corrida anterior que estorbe?

Por qué ahora. Los números del panel solo salen iguales si la tabla parte vacía.

En el almacén. Es sacar del estante la libreta vieja antes de empezar una nueva.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` hace que no reclame si la tabla no existe.
- `panel_lab12` es el nombre de la tabla.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS panel_lab12
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se borró, o no existía. En los dos casos sale lo mismo.

Celda 0.3 · Enero de un tirón

La pregunta. ¿Cómo queda la tabla después de la carga inicial?

Por qué ahora. La primera carga de casi toda tabla es el histórico entero, mucho más gordo que lo que llega después. Ese es el primer achaque, y la pregunta del paso 5 lo va a encontrar.

En el almacén. Es abrir la libreta y pegar de una vez todos los papelitos de enero, en un solo cajón.

La sentencia, parte por parte.

- `CREATE TABLE panel_lab12` crea la tabla.
- `USING iceberg` hace que sea una tabla Iceberg.
- `PARTITIONED BY (months(fecha_emision))` guarda los documentos en un cajón por mes. `months` toma la fecha de emisión y calcula a qué mes pertenece, sin que tengas que crear una columna con el mes.
- `AS SELECT * FROM curso.dte_2024` crea la tabla con las columnas de esa consulta y la llena con su resultado. `curso.dte_2024` es la tabla de referencia con los documentos de 2024, en el espacio `curso`.
- `WHERE fecha_emision BETWEEN DATE '2024-01-01' AND DATE '2024-01-31'` se queda solo con enero. `DATE '...'` convierte el texto en fecha, y `BETWEEN` incluye los dos extremos.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE panel_lab12
USING iceberg
PARTITIONED BY (months(fecha_emision))
AS SELECT * FROM curso.dte_2024
WHERE fecha_emision BETWEEN DATE '2024-01-01' AND DATE '2024-01-31'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla existe y tiene enero adentro, en un solo cajón.

Celda 0.4 · Veintidós cargas chicas

La pregunta. ¿Qué deja en la tabla una ingesta que carga un poco cada día?

Por qué ahora. Así se cargan las tablas de verdad, de a poco, todos los días. Cada carga deja su papelito y su página, y ese es el segundo achaque.

En el almacén. Es anotar cada día solo lo que llegó ese día, en un papelito aparte.

Esta celda viene escrita en el cuaderno y es la única que no es SQL. Son veintidós escrituras, y teclearlas una por una no enseña nada.

La sentencia, parte por parte.

- `# Celda 0.4` es un comentario de Python.
- `ESPACIO = "mi_espacio"` guarda el nombre de tu espacio en una variable, para usarlo más abajo.
- `for mes in range(2, 13):` recorre los meses del 2 al 12. `range(2, 13)` entrega los números desde el 2 hasta el 12, porque el último número no se incluye.
- `for dia in (10, 20):` recorre los dos días de cada mes, el 10 y el 20.
- `fecha = f"2024-{mes:02d}-{dia:02d}"` arma la fecha como texto. La `f` antes de las comillas permite meter variables entre llaves, y `:02d` escribe el número con dos dígitos, con un cero adelante si hace falta. El mes 2 queda como `02`.
- `spark.sql(...)` manda una sentencia SQL a Spark desde Python. `spark` es la conexión a Spark, que creó la celda 0.1.
- `f"INSERT INTO {ESPACIO}.panel_lab12 "` y la línea siguiente arman la sentencia. `INSERT INTO` agrega filas a la tabla, y `SELECT * FROM curso.dte_2024 WHERE fecha_emision = DATE '{fecha}'` trae los documentos de ese día.
- `print("cargado", fecha)` avisa en pantalla cada carga.
- `print("listo: 22 cargas diarias")` avisa al final.

```
python
# Celda 0.4
ESPACIO = "mi_espacio"

for mes in range(2, 13):
    for dia in (10, 20):
        fecha = f"2024-{mes:02d}-{dia:02d}"
        spark.sql(
            f"INSERT INTO {ESPACIO}.panel_lab12 "
            f"SELECT * FROM curso.dte_2024 WHERE fecha_emision = DATE '{fecha}'"
        )
        print("cargado", fecha)

print("listo: 22 cargas diarias")
```

Lo que sale en pantalla.

```
text
cargado 2024-02-10
cargado 2024-02-20
cargado 2024-03-10
cargado 2024-03-20
cargado 2024-04-10
cargado 2024-04-20
cargado 2024-05-10
cargado 2024-05-20
cargado 2024-06-10
cargado 2024-06-20
cargado 2024-07-10
cargado 2024-07-20
cargado 2024-08-10
cargado 2024-08-20
cargado 2024-09-10
cargado 2024-09-20
cargado 2024-10-10
cargado 2024-10-20
cargado 2024-11-10
cargado 2024-11-20
cargado 2024-12-10
cargado 2024-12-20
listo: 22 cargas diarias
```

Cómo se lee. Veintidós líneas `cargado`, una por día, del 10 de febrero al 20 de diciembre, y el aviso final. Cada línea es una escritura, y cada escritura dejó un papelito y una página.

Celda 0.5 · Primera corrección

La pregunta. ¿Qué hace una corrección con los papelitos?

Por qué ahora. Es el tercer achaque. Para cambiar filas de un papelito, Iceberg no tacha. Escribe un papelito nuevo con el contenido ya corregido y deja de usar el viejo, que queda en el suelo.

En el almacén. Es copiar en limpio el papelito de enero con el renglón corregido y dejar el viejo en el cajón de los descartes.

La sentencia, parte por parte.

- `UPDATE panel_lab12` corrige filas de la tabla.
- `SET estado_sii = 'ACEPTADO'` deja el estado en aceptado.
- `WHERE fecha_emision BETWEEN DATE '2024-01-01' AND DATE '2024-01-31'` se limita a enero.
- `AND estado_sii = 'EN_REVISION'` y dentro de enero, solo a los documentos que estaban en revisión.

```
sql
%%sql
-- Celda 0.5
UPDATE panel_lab12
SET estado_sii = 'ACEPTADO'
WHERE fecha_emision BETWEEN DATE '2024-01-01' AND DATE '2024-01-31'
AND estado_sii = 'EN_REVISION'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La sentencia terminó. El cuaderno no dice cuántas filas cambió. En la sección de preguntas frecuentes está lo que de verdad hizo esta corrección.

Celda 0.6 · Segunda corrección

La pregunta. ¿Cómo se corrige un monto que faltaba?

Por qué ahora. A los documentos del 5 de enero les faltaba el IVA. Es otra corrección sobre el mismo papelito de enero.

En el almacén. Es volver a copiar en limpio el papelito de enero, esta vez con el IVA del 5 de enero.

La sentencia, parte por parte.

- `UPDATE panel_lab12` corrige filas.
- `SET monto_iva = round(monto_neto * 0.19)` calcula el IVA como el 19 % del neto. `round` lo redondea al entero.
- `WHERE fecha_emision = DATE '2024-01-05'` se limita al 5 de enero.

```
sql
%%sql
-- Celda 0.6
UPDATE panel_lab12
SET monto_iva = round(monto_neto * 0.19)
WHERE fecha_emision = DATE '2024-01-05'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La corrección entró. El papelito de enero se reescribió y el anterior quedó en el suelo.

Celda 0.7 · Tercera corrección

La pregunta. ¿Qué pasa si se corrige el tipo de documento?

Por qué ahora. El primer día del año quedó con el tipo equivocado. Es la tercera corrección sobre enero.

En el almacén. Otra copia en limpio del papelito de enero.

La sentencia, parte por parte.

- `UPDATE panel_lab12` corrige filas.
- `SET tipo_dte = 33` cambia el tipo a 33, que es la factura afecta.
- `WHERE fecha_emision = DATE '2024-01-01'` se limita al 1 de enero.
- `AND tipo_dte = 34` y dentro de ese día, a los documentos de tipo 34, que es la factura exenta.

```
sql
%%sql
-- Celda 0.7
UPDATE panel_lab12
SET tipo_dte = 33
WHERE fecha_emision = DATE '2024-01-01'
AND tipo_dte = 34
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La corrección entró, con otra copia del papelito de enero.

Celda 0.8 · Una columna nueva

La pregunta. ¿Qué le pasa a la tabla cuando se le agrega una columna?

Por qué ahora. Aparece una exigencia que no existía cuando se creó la tabla, dejar registrado quién revisó cada documento. Es el cuarto achaque, un cambio de estructura, y la pregunta del paso 7 lo va a encontrar.

En el almacén. Es agregar una casilla nueva en la libreta. Las páginas que ya estaban no se reescriben, la casilla queda en blanco en ellas.

La sentencia, parte por parte.

- `ALTER TABLE panel_lab12` cambia la definición de la tabla.
- `ADD COLUMN revisado_por STRING` agrega la columna `revisado_por`, de texto.

```
sql
%%sql
-- Celda 0.8
ALTER TABLE panel_lab12 ADD COLUMN revisado_por STRING
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La columna existe. No se movió ni un dato. Los documentos que ya estaban la tienen vacía.

Celda 0.9 · Un borrado

La pregunta. ¿Qué hace la libreta cuando se borra un día completo?

Por qué ahora. Los documentos del 20 de julio se cargaron dos veces y hay que sacarlos. Es el último achaque, y la pregunta del paso 6 lo va a encontrar.

En el almacén. Ese día entró en una sola carga, así que sus documentos están todos en el mismo papelito y en ningún otro. Para borrarlos, la libreta no escribe nada nuevo. Le basta con dejar de nombrar ese papelito.

La sentencia, parte por parte.

- `DELETE FROM panel_lab12` borra filas de la tabla.
- `WHERE fecha_emision = DATE '2024-07-20'` se limita al 20 de julio.

```
sql
%%sql
-- Celda 0.9
DELETE FROM panel_lab12 WHERE fecha_emision = DATE '2024-07-20'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. El borrado entró. Como no hubo que reescribir nada, la página queda registrada como un borrado, `delete`, y no como una reescritura.

Paso 1 · ¿Cuántos archivos tiene la tabla y cuánto pesan?

Celda 1.1 · Archivos y peso

La pregunta. ¿En cuántos pedazos está la tabla y cuánto ocupa?

Por qué ahora. Es la primera pregunta de cualquier panel. Muchos archivos para pocos datos es la señal de que hace falta compactar.

En el almacén. Es contar los papelitos que están en uso y sumar lo que pesan, leyendo la libreta y no los papelitos.

La sentencia, parte por parte.

- `count(*) AS archivos` cuenta los archivos. En la vista `.files` cada fila es un papelito vigente.
- `sum(record_count) AS filas` suma las filas de todos los papelitos. `record_count` es cuántas filas trae cada uno.
- `sum(file_size_in_bytes) AS bytes` suma lo que pesan. `file_size_in_bytes` es el peso de cada papelito en bytes.
- `FROM mi_espacio.panel_lab12.files` es la vista de sistema con los papelitos vigentes. Se nombra con tres partes, el espacio, la tabla y la vista, aunque ya hayas hecho `USE`.

```
sql
%%sql
-- Celda 1.1
SELECT count(*)           AS archivos,
       sum(record_count)   AS filas,
       sum(file_size_in_bytes) AS bytes
FROM mi_espacio.panel_lab12.files
```

Lo que sale en pantalla.

```
text
archivos  filas  bytes
22        4232  208727
1 fila.
```

Cómo se lee. La tabla tiene 22 archivos, 4232 filas y pesa 208727 bytes, unos 0,21 MB.

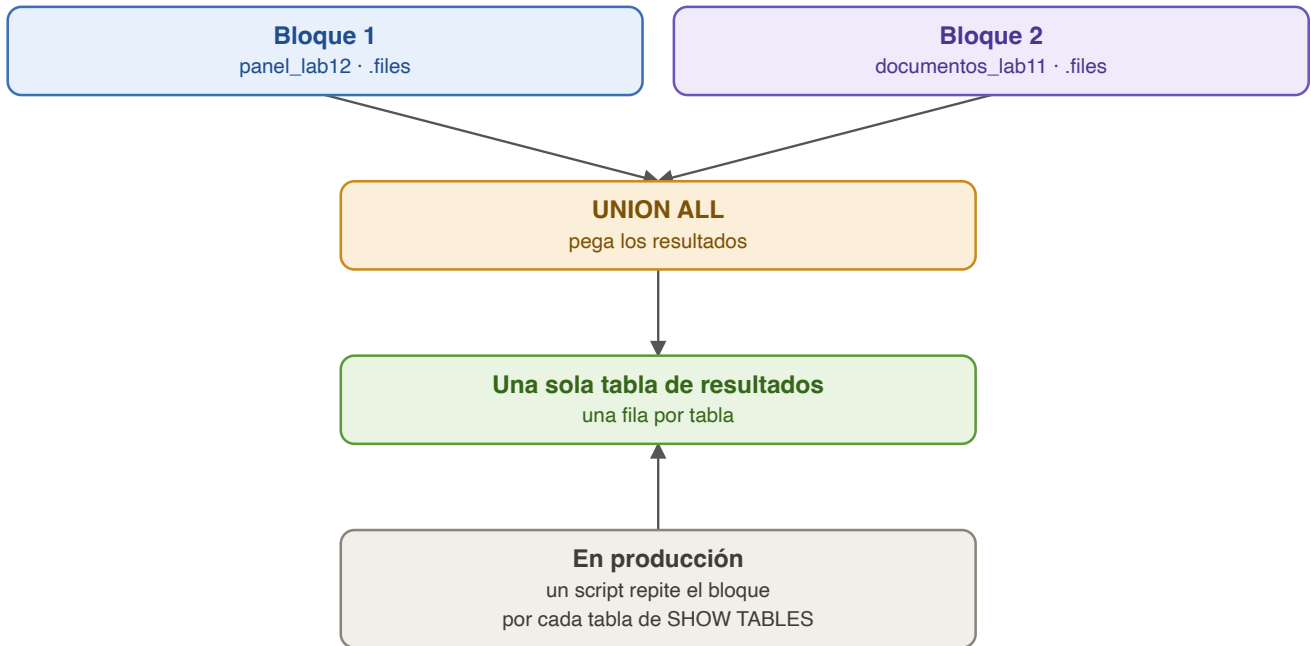
Son 22 archivos para un poco más de cuatro mil filas. En promedio, menos de doscientas filas por archivo. Es una tabla chica partida en muchos pedazos, que es justo lo que la ingesta diaria deja.

Celda 1.2 · La misma pregunta para varias tablas

La pregunta. ¿Cómo se hace la pregunta anterior sobre varias tablas a la vez?

Por qué ahora. Un panel no mira una tabla, mira todas. Esta celda muestra la forma que toma la consulta en una plataforma, con dos tablas de tu espacio.

En el almacén. Es pasar por dos estantes, contar los papelitos de cada uno y anotar los dos números en la misma hoja.



Bloque 1 (azul) y **Bloque 2** (morado). Cada bloque es la misma consulta de conteo sobre una tabla distinta, con una columna fija que dice de qué tabla se trata.

UNION ALL (amarillo). Pega el resultado de los dos bloques uno debajo del otro. **ALL** significa que no quita filas repetidas.

Una sola tabla de resultados (verde). Queda una fila por tabla, ordenada de la que tiene más archivos a la que tiene menos.

En producción (gris). Nadie teclea un bloque por tabla. Un script lee la lista de tablas y arma la consulta repitiendo el bloque por cada una.

La sentencia, parte por parte.

- `SELECT 'panel_lab12' AS tabla` pone en cada fila un texto fijo con el nombre de la tabla, en una columna llamada `tabla`.
- `count(*) AS archivos` cuenta los papelitos vigentes de esa tabla.
- `FROM mi_espacio.panel_lab12.files` es la vista de papelitos de `panel_lab12`.
- `UNION ALL` pega el resultado del bloque siguiente debajo del primero.
- `SELECT 'documentos_lab11' AS tabla, count(*) AS archivos FROM mi_espacio.documentos_lab11.files` es el mismo bloque sobre otra tabla de tu espacio, `documentos_lab11`.
- `ORDER BY archivos DESC` ordena de más archivos a menos. `DESC` es descendente.

```
sql
%%sql
-- Celda 1.2
SELECT 'panel_lab12' AS tabla, count(*) AS archivos
FROM mi_espacio.panel_lab12.files
UNION ALL
SELECT 'documentos_lab11' AS tabla, count(*) AS archivos
FROM mi_espacio.documentos_lab11.files
ORDER BY archivos DESC
```

Lo que sale en pantalla.

```
text
tabla      archivos
panel_lab12 22
documentos_lab11 1
2 filas.
```

Cómo se lee. `panel_lab12` tiene 22 archivos y `documentos_lab11` tiene 1. La primera está fragmentada y la segunda no.

La consulta tiene un precio. Si una sola de las tablas nombradas no existiera, falla la consulta entera y no devuelve ninguna fila. Por eso el script que la arma tiene que leer la lista de tablas de verdad, y no traerla escrita. Si en tu espacio no está `documentos_lab11`, esta celda falla por eso mismo.

Paso 2 · ¿Cuántos archivos chicos hay?

Celda 2.1 · Archivos chicos

La pregunta. ¿Cuántos de los archivos son chicos, y cuánto pesa el más chico?

Por qué ahora. Un archivo chico cuesta casi lo mismo de abrir que uno grande. Hay que ir a buscarlo, abrirlo y leer su pie antes de mirar una sola fila. Muchos archivos chicos son muchas aperturas, y ahí se va el tiempo de las consultas. Esta pregunta no los junta, los cuenta para saber si hace falta juntarlos.

En el almacén. Es separar los papelitos que tienen apenas un par de renglones de los que están llenos.

La sentencia, parte por parte.

- `count(*) AS archivos` cuenta todos los papelitos vigentes.
- `count(CASE WHEN file_size_in_bytes < 1048576 THEN 1 END) AS chicos` cuenta solo los chicos. `CASE WHEN ... THEN 1 END` pone un 1 a cada papelito que pesa menos que el umbral y deja vacío al resto, y `count` cuenta solo los que no quedaron vacíos. El umbral es `1048576`, en bytes, que son 1,05 MB.
- `min(file_size_in_bytes) AS el_mas_chico` es el peso del papelito más liviano.
- `FROM mi_espacio.panel_lab12.files` es la vista de papelitos vigentes.

```
sql
%%sql
-- Celda 2.1
SELECT count(*)                AS archivos,
       count(CASE WHEN file_size_in_bytes < 1048576 THEN 1 END) AS chicos,
       min(file_size_in_bytes) AS el_mas_chico
FROM mi_espacio.panel_lab12.files
```

Lo que sale en pantalla.

```
text
archivos  chicos  el_mas_chico
22        22      5887
1 fila.
```

Cómo se lee. Los 22 archivos son chicos, y el más chico pesa 5887 bytes, menos de un centésimo de MB.

El umbral de un poco más de un MB es de este laboratorio, donde todo es chico. En una plataforma con archivos de cientos de MB el umbral sería mucho más alto. Aquí la respuesta es la peor posible, todos los archivos están bajo el umbral.

Paso 3 · ¿Cuántas páginas acumula la tabla?

Celda 3.1 · Páginas y fechas

La pregunta. ¿Cuánta historia arrastra la tabla, y desde cuándo?

Por qué ahora. La fecha de la página más vieja dice hasta dónde hacia atrás se puede preguntar hoy, y también cuánta historia habría que botar si el disco aprieta.

En el almacén. Es contar las páginas de la libreta y mirar la fecha de la primera y de la última.

La sentencia, parte por parte.

- `count(*) AS paginas` cuenta las páginas. En la vista `.snapshots` cada fila es una página.
- `min(committed_at) AS la_mas_vieja` es la hora de la página más vieja. `committed_at` es cuándo se colgó cada página, en UTC.
- `max(committed_at) AS la_mas_nueva` es la hora de la más nueva.
- `FROM mi_espacio.panel_lab12.snapshots` es la vista de sistema con las páginas.

```
sql
%%sql
-- Celda 3.1
SELECT count(*)          AS paginas,
       min(committed_at) AS la_mas_vieja,
       max(committed_at) AS la_mas_nueva
FROM mi_espacio.panel_lab12.snapshots
```

Lo que sale en pantalla.

```
text
paginas  la_mas_vieja          la_mas_nueva
27       2026-09-25 18:09:57.685  2026-09-25 18:10:21.208
1 fila.
```

Cómo se lee. La tabla tiene 27 páginas, la más vieja de las 18:09:57.685 UTC y la más nueva de las 18:10:21.208 UTC del 25 de septiembre de 2026, las 15:10 en Chile.

Las 27 páginas son la carga de enero, las veintidós cargas diarias, las tres correcciones y el borrado. El `ADD COLUMN` no deja página, porque no cambia datos. Toda la historia cabe en menos de medio minuto porque se armó en el paso 0. En una tabla real, la distancia entre la más vieja y la más nueva es lo que importa.

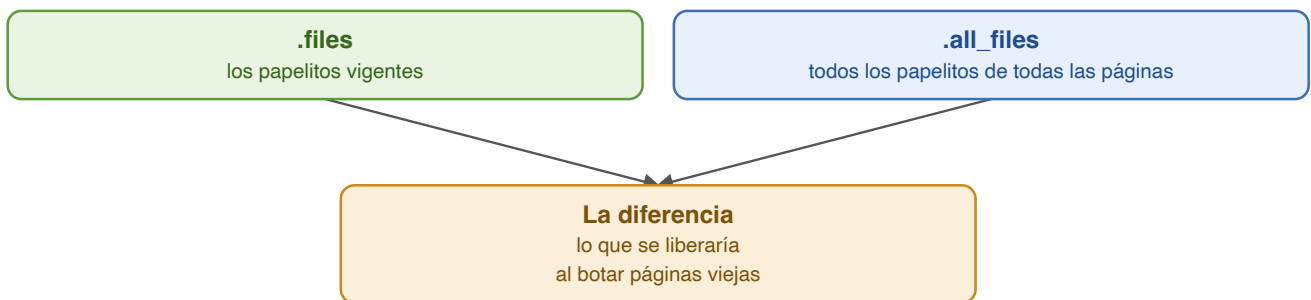
Paso 4 · ¿Cuánto ocupan las páginas que ya no cuentan?

Celda 4.1 · Lo que se liberaría

La pregunta. ¿Cuánto espacio ocupan los archivos que ya nadie consulta?

Por qué ahora. Las correcciones y el borrado dejaron papelitos que la página de hoy ya no nombra. No se borran, porque las páginas anteriores todavía los nombran, y mientras esas páginas existan se puede pedir la tabla como estaba antes.

En el almacén. Es comparar los papelitos que están en uso con todos los que hay en la bodega, contando los descartes.



.files (verde). Son los papelitos que nombra la página vigente. Es lo que responde una consulta hoy.

.all_files (azul). Son los papelitos que nombra cualquier página, la de hoy y las anteriores.

La diferencia (amarillo). Lo que está en la segunda y no en la primera es lo que ocupan los papelitos viejos. Es lo que se recuperaría al botar las páginas que los nombran.

La sentencia, parte por parte.

- `(SELECT count(*) FROM mi_espacio.panel_lab12.files) AS archivos_vigentes` cuenta los papelitos vigentes. Cada `(SELECT ...)` entre paréntesis es una consulta dentro de otra, que devuelve un solo número.
- `(SELECT count(*) FROM mi_espacio.panel_lab12.all_files) AS archivos_totales` cuenta todos los papelitos.
- `(SELECT sum(file_size_in_bytes) FROM mi_espacio.panel_lab12.all_files) - (SELECT sum(file_size_in_bytes) FROM mi_espacio.panel_lab12.files) AS bytes_recuperables` resta el peso de los vigentes al peso de todos.
- El `SELECT` de afuera no tiene `FROM`, porque solo junta los tres números en una fila.

```
sql
%%sql
-- Celda 4.1
SELECT (SELECT count(*) FROM mi_espacio.panel_lab12.files) AS archivos_vigentes,
       (SELECT count(*) FROM mi_espacio.panel_lab12.all_files) AS archivos_totales,
       (SELECT sum(file_size_in_bytes) FROM mi_espacio.panel_lab12.all_files)
       - (SELECT sum(file_size_in_bytes) FROM mi_espacio.panel_lab12.files) AS bytes_recuperables
```

Lo que sale en pantalla.

```
text
archivos_vigentes  archivos_totales  bytes_recuperables
22                25                152322
1 fila.
```

Cómo se lee. Hay 22 archivos vigentes y 25 en total, así que tres papelitos ya no cuentan. Ocupan 152322 bytes, unos 0,15 MB.

Los tres papelitos viejos son dos versiones anteriores del papelito de enero, que dejaron las correcciones, y el papelito del 20 de julio, que dejó el borrado. Lo que recuperarías es casi tres cuartos de lo que pesa la tabla vigente, que eran 208727 bytes. El umbral de ejemplo es preocuparse cuando lo recuperable pasa del doble de lo vigente.

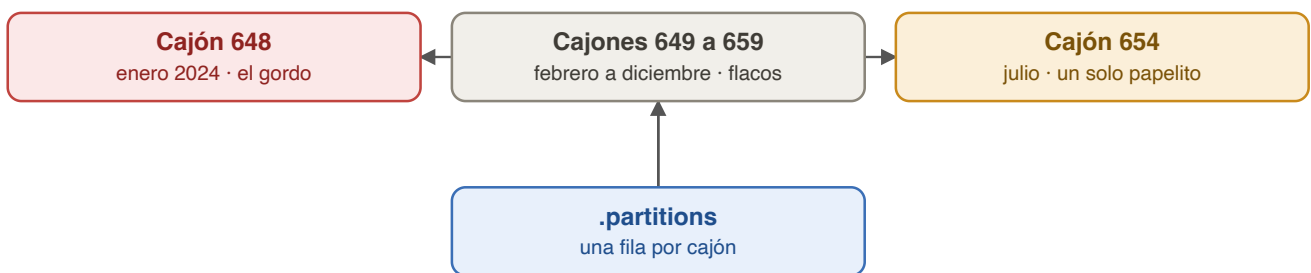
Paso 5 · ¿Hay particiones desparejas?

Celda 5.1 · Los cajones

La pregunta. ¿Hay un cajón con muchas más filas que los demás?

Por qué ahora. Esta pregunta solo aplica a tablas particionadas, y `panel_lab12` está particionada por mes. Un cajón desbalanceado duele por dos razones. La consulta que cae en él tarda mucho más que las otras, y cuando hay que reprocesarlo, el trabajo no se reparte.

En el almacén. Es mirar los doce cajones del mes y ver si alguno está reventando mientras los otros están casi vacíos.



Cajón 648 (rojo). Es enero de 2024, cargado de un tirón. Tiene todos los documentos del mes en un solo papelito.

Cajones 649 a 659 (gris). Son febrero a diciembre, con dos días cargados cada uno. Tienen pocas filas y dos papelitos.

Cajón 654 (amarillo). Es julio. Tiene un solo papelito porque el del 20 de julio se dejó de nombrar con el borrado.

.partitions (azul). Es la vista de sistema con una fila por cajón, con cuántas filas y cuántos papelitos tiene cada uno.

La sentencia, parte por parte.

- `partition` es el valor que identifica al cajón.
- `record_count AS filas` es cuántas filas tiene el cajón.
- `file_count AS archivos` es cuántos papelitos tiene.
- `FROM mi_espacio.panel_lab12.partitions` es la vista de cajones.
- `ORDER BY record_count DESC` ordena del cajón más gordo al más flaco.

```
sql
%%sql
-- Celda 5.1
SELECT partition,
       record_count AS filas,
       file_count   AS archivos
FROM mi_espacio.panel_lab12.partitions
ORDER BY record_count DESC
```

Lo que sale en pantalla.

```
text
partition  filas  archivos
(648,)     2500   1
(655,)     188   2
(658,)     181   2
(653,)     174   2
(651,)     169   2
(649,)     165   2
(657,)     165   2
(650,)     158   2
(659,)     157   2
(656,)     143   2
(652,)     142   2
(654,)     90    1
12 filas.
```

Cómo se lee. El cajón (648,) tiene 2500 filas en 1 archivo. Los otros once tienen entre 90 y 188 filas.

La primera columna no dice 2024-01, dice un número. Es cuántos meses pasaron desde enero de 1970, que es como Iceberg guarda por dentro un cajón mensual. Seiscientos cuarenta y ocho meses son cincuenta y cuatro años justos, así que 648 es enero de 2024, 649 febrero y así hasta 659, que es diciembre. Los paréntesis y la coma son la forma de escribir un grupo de valores con uno solo adentro, porque una tabla puede estar particionada por más de una columna.

El cajón (654,) , julio, tiene 90 filas y 1 archivo. Es el mes del borrado, y le quedó solo el papelito del 10 de julio.

Para saber si está desparejo, se compara el más gordo con la mediana, que es el valor del medio cuando se ordenan todos. Aquí la mediana es 165, y 2500 filas son unas quince veces eso. Con el umbral de ejemplo, diez veces la mediana, la tabla está despareja.

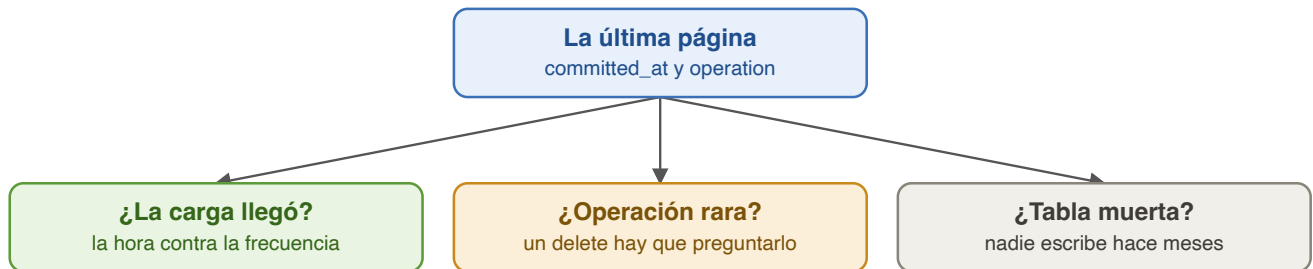
Paso 6 · ¿Cuándo fue la última escritura?

Celda 6.1 · La última página

La pregunta. ¿Cuándo escribió alguien por última vez en la tabla, y qué hizo?

Por qué ahora. Sirve para saber si la carga de anoche entró, para ver una operación que no corresponde y para encontrar tablas que nadie usa.

En el almacén. Es mirar la fecha de la última página de la libreta y qué dice que se hizo.



La última página (azul). Es la página más reciente, con su hora y su operación.

¿La carga llegó? (verde). Es el uso principal. Si la tabla se carga todas las noches y la última escritura es de hace dos días, la carga falló, aunque nadie haya visto un error.

¿Operación rara? (amarillo). Un `delete` o un `overwrite` en una tabla que solo recibe cargas es algo que hay que ir a preguntar hoy.

¿Tabla muerta? (gris). Si nadie escribe hace meses, la tabla puede estar ocupando disco sin servirle a nadie.

La sentencia, parte por parte.

- `committed_at AS ultima_escritura` es la hora en que se colgó la página, en UTC.
- `operation AS que_hizo` es qué se hizo en esa página.
- `FROM mi_espacio.panel_lab12.snapshots` es la vista de páginas.
- `ORDER BY committed_at DESC` ordena de la más nueva a la más vieja.
- `LIMIT 1` se queda solo con la primera, que es la más nueva.

```
sql
%%sql
-- Celda 6.1
SELECT committed_at AS ultima_escritura,
       operation      AS que_hizo
FROM   mi_espacio.panel_lab12.snapshots
ORDER BY committed_at DESC
LIMIT 1
```

Lo que sale en pantalla.

```
text
ultima_escritura      que_hizo
2026-09-25 18:10:21.208 delete
1 fila.
```

Cómo se lee. La última escritura fue a las 18:10:21.208 UTC, las 15:10 en Chile, y fue un `delete`, el borrado del 20 de julio.

Es la misma hora que `la_mas_nueva` de la celda 3.1, como corresponde. En una tabla de cargas, un `delete` como última operación es justo lo que hay que ir a preguntar.

Paso 7 · ¿Cuántas versiones de portada hay?

Celda 7.1 · Portadas y esquema

La pregunta. ¿Alguien cambió la estructura de la tabla?

Por qué ahora. Es lo primero que uno quiere saber cuando un reporte que funcionaba deja de funcionar.

En el almacén. Cada vez que algo cambia, la libreta estrena una portada nueva que describe cómo es la libreta en ese momento. Contar las portadas y ver el número de la última dice si alguien cambió las casillas.

La sentencia, parte por parte.

- `count(*) AS portadas` cuenta las portadas. En la vista `.metadata_log_entries` cada fila es un `metadata.json`, una portada.
- `max(latest_schema_id) AS ultimo_esquema` es el número de versión de la estructura en la portada más nueva. `latest_schema_id` es el identificador del esquema, es decir, de la lista de columnas con sus tipos.
- `FROM mi_espacio.panel_lab12.metadata_log_entries` es la vista con la lista de portadas.

```
sql
%%sql
-- Celda 7.1
SELECT count(*)           AS portadas,
       max(latest_schema_id) AS ultimo_esquema
FROM mi_espacio.panel_lab12.metadata_log_entries
```

Lo que sale en pantalla.

```
text
portadas  ultimo_esquema
28        1
1 fila.
```

Cómo se lee. Hay 28 portadas y el último esquema es el 1. La tabla nació con el esquema 0, así que alguien cambió su estructura una vez.

Son 28 portadas y no 27 como las páginas porque el `ADD COLUMN` escribió una portada nueva sin escribir una página. Ese cambio es el que subió el esquema de 0 a 1.

Paso 8 · ¿La tabla se compactó alguna vez?

Celda 8.1 · Archivos y compactaciones

La pregunta. ¿Hay muchos archivos y nadie los ha juntado nunca?

Por qué ahora. Cierra el círculo del panel. Compactar con `rewrite_data_files` deja siempre una página de tipo `replace`. Una tabla con muchos archivos y ninguna página `replace` es una tabla que nadie ha compactado.

En el almacén. Es ver si alguna vez alguien pasó en limpio los papelitos chicos.

La sentencia, parte por parte.

- `(SELECT count(*) FROM mi_espacio.panel_lab12.files) AS archivos` cuenta los papelitos vigentes.
- `(SELECT count(*) FROM mi_espacio.panel_lab12.snapshots WHERE operation = 'replace') AS veces_compactada` cuenta las páginas de tipo `replace`.

```
sql
%%sql
-- Celda 8.1
SELECT (SELECT count(*) FROM mi_espacio.panel_lab12.files) AS
archivos,
      (SELECT count(*) FROM mi_espacio.panel_lab12.snapshots WHERE operation = 'replace') AS
veces_compactada
```

Lo que sale en pantalla.

```
text
archivos  veces_compactada
22        0
1 fila.
```

Cómo se lee. 22 archivos y ninguna compactación. Con muchos archivos chicos y cero `replace`, esta tabla necesita que alguien la compacte.

Preguntas frecuentes

¿El UNION ALL de la celda 1.2 consulta solo dos tablas?

Sí. Consulta exactamente las dos que nombra, `panel_lab12` y `documentos_lab11`, y ninguna más. Para mirar todas, en producción no se tecléa. Un script lee la lista con `SHOW TABLES IN mi_espacio`, arma un bloque `SELECT 'nombre' AS tabla, count(*) AS archivos FROM mi_espacio.nombre.files` por cada tabla que encuentra, los pega con `UNION ALL` y corre la consulta. Como lee la lista de verdad, nunca nombra una tabla que no existe, que es lo que haría fallar la consulta entera.

¿De dónde salen los umbrales del cierre?

Son de ejemplo, puestos para que se pueda conversar sobre los números de este laboratorio. No son reglas de Iceberg. Iceberg no dice en ninguna parte que cien páginas sean muchas ni que un MB sea chico. La consulta entrega el número y el umbral lo pone el administrador, mirando lo suyo. Cien páginas no dicen nada por sí solas. Una tabla que se carga cada quince minutos junta casi cien en un día y está sana, y una que se carga una vez al mes con cien páginas guarda más de ocho años de historia.

¿Cómo se calcula el mínimo y el máximo de retención en un trabajo real?

El mínimo lo pone el negocio. Es cuánto hacia atrás hay que poder volver. Se calcula como el tiempo que se tarda en darse cuenta de que una carga salió mala, más el tiempo en corregirla, más un margen. Si la tabla se carga cada día y los errores aparecen en una revisión semanal, un mínimo razonable son catorce días.

El máximo lo pone el disco. Cada página guarda lo que agregó. El costo de la historia es lo que agrega cada carga, por las cargas de cada día, por los días de retención. Eso se mide mirando cuánto agrega cada página en `.snapshots`.

Los hitos largos van con etiquetas. Si una auditoría pide cómo estaba la tabla al cierre de un año, no se guardan años de páginas. Se le pone una etiqueta a la página del cierre con `CREATE TAG`, por ejemplo `ALTER TABLE panel_lab12 CREATE TAG cierre_2024`, y `expire_snapshots` no bota las páginas etiquetadas.

Se deja escrito y se programa. La retención se guarda como propiedad de la tabla, `history.expire.max-snapshot-age-ms`, que por defecto son cinco días, y un `expire_snapshots` programado cada noche la aplica. Si nadie lo corre, no se bota nada.

¿Para qué sirve mirar la última escritura?

Sobre todo para saber si la carga de anoche corrió. Si un proceso nocturno falla sin avisar, la tabla sigue ahí, las consultas funcionan y los reportes salen, pero con los datos de anteayer y sin ningún error. Si la tabla se carga todas las noches y su última escritura es de hace dos días, la carga está atrasada, y el panel lo muestra antes de que alguien decida con datos viejos. Los otros dos usos son ver una operación que no corresponde, como el `delete` de hoy, y encontrar tablas que nadie escribe hace meses.

¿Qué es el número de la partición, el 648?

Es el mes guardado como un número, contado desde enero de 1970. Enero de 1970 es el 0, y cada mes suma uno. Enero de 2024 está cincuenta y cuatro años después, y cincuenta y cuatro por doce son 648. Por eso `648` es enero de 2024 y `659` diciembre de 2024. Iceberg lo guarda así porque un número se compara y se ordena más rápido que una fecha.

¿Por qué tres correcciones dejaron solo dos papelitos viejos de enero?

Porque la primera corrección, la de la celda 0.5, no encontró filas. Ningún documento de enero estaba en revisión. Igual dejó una página, de tipo `overwrite`, pero no reescribió ningún papelito. Las otras dos sí encontraron filas y cada una dejó una versión vieja del papelito de enero en el suelo.

¿Estas consultas leen datos?

No. Todas leen la portada y los manifiestos, lo que la libreta anota de sí misma. Por eso son rápidas en una tabla de cuatro mil filas y en una de miles de millones, y se pueden correr todas las mañanas sobre todas las tablas.

¿Por qué las horas no calzan con mi reloj?

Porque Spark muestra las horas en UTC, la hora universal. Chile en septiembre está tres horas atrás, así que las 18:10 UTC son las 15:10 en Chile.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Ocho preguntas cortas que juntas dicen si una tabla está sana. Ninguna abre un solo papelito, y por eso se pueden correr todos los días.

En el almacén. El dueño ya no revisa caja por caja. Mira la libreta cada mañana, y en un minuto sabe si hay demasiados papelitos chicos, descartes ocupando lugar, un cajón reventando, una carga que no llegó o una casilla nueva que nadie avisó.

Lo que salió en panel_lab12, con un umbral de ejemplo para cada pregunta.

#	La pregunta	Lo que salió	Cuándo preocuparse, de ejemplo
1	Archivos y peso	22 archivos, 4232 filas, 208727 bytes	muchos archivos para pocas filas
2	Archivos chicos	22 de 22, el más chico de 5887 bytes	más de la mitad bajo el umbral
3	Páginas acumuladas	27 páginas	más de cien, o la más vieja de hace meses
4	Espacio de páginas viejas	25 archivos en total contra 22 vigentes, 152322 bytes	si pasa del doble de lo vigente
5	Particiones desparejas	enero con 2500 filas, el resto entre 90 y 188	si una tiene diez veces la mediana
6	Última escritura	un <code>delete</code>	si no hay ninguna hace más de lo que dice su frecuencia de carga
7	Versiones de portada	28 portadas, esquema 1	si el esquema cambió y nadie avisó
8	Sin compactar	22 archivos, 0 <code>replace</code>	muchos archivos y cero <code>replace</code>

Ninguno de esos umbrales es una regla de Iceberg. Cada plataforma pone los suyos mirando el tamaño de sus tablas, cada cuánto se cargan, cuánta historia exige su auditoría y cuánto disco tiene. Lo que no cambia es la idea. Ninguna de estas consultas lee datos, y por eso se pueden mirar todos los días.